

Deep and Un-Deep Visual Inertial Odometry

Shibani Senthilbabu
RBE Department
Worcester Polytechnic Institute
ssenthilbabu@wpi.edu

Ajay Adit Jagan
RBE Department
Worcester Polytechnic Institute
ajagan@wpi.edu

Abstract—We present a complete pipeline for learning-based visual-inertial odometry trained entirely on synthetic data. A Blender-based data generator produces randomized drone-like trajectories over textured ground planes, rendering RGB images at 100 Hz alongside simulated noisy IMU readings. We train three architectures — vision-only (ResNet18 backbone), inertial-only (1-D CNN), and a late-fusion visual-inertial network — to regress frame-to-frame relative poses. All models are evaluated by dead-reckoning integration over held-out test sequences followed by SE(3) alignment and RMSE-ATE computation.

I. SYNTHETIC DATA GENERATION

Because real VIO datasets are scarce and sensor-specific, we generate all training and test data synthetically using Blender and a custom IMU noise model. The pipeline has four stages: trajectory generation, IMU simulation, Blender rendering, and dataset packaging.

A. Trajectory Generator

Each sequence is a 10-second flight at 1 kHz internal resolution. We sample n random 3-D waypoints in a bounded volume (± 3 m horizontally, 5–10 m altitude) and interpolate them with clamped cubic splines to produce smooth position curves. Velocity is obtained analytically from the spline derivative. Orientations are constructed from Euler angles: yaw follows the smoothed velocity heading, while roll and pitch are independent cubic splines with random control points clamped to $\pm 45^\circ$. The quaternion sequence is converted to body-frame angular velocity via finite differences and light smoothing.

B. IMU Simulation

Realistic noise is added using the IMU error model from gnss-ins-sim (via OysterSim), which applies a first-order Gauss–Markov bias drift.

C. Blender Rendering

Camera poses are subsampled from the 1 kHz trajectory at 100 Hz (every 10th sample), yielding 1000 frames per sequence. A single textured ground plane serves as the scene. To ensure visual diversity across sequences, each run randomly selects one of 35 textures — 13 procedural (checkerboard, brick, concrete, marble, wood, grass, etc.) and 22 OpenStreetMap city-map tiles (Manhattan, Paris, Tokyo, and others) stitched from zoom-16 tiles into 1024×1024 images. The plane size and UV tiling scale are also randomized per sequence. Rendering uses Blender’s Workbench engine in flat-texture mode for speed, producing 320×240 RGB frames. Camera intrinsics are saved once per sequence.

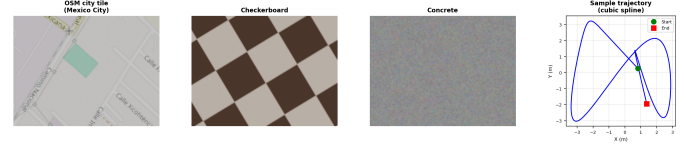


Fig. 1. Sample dataset outputs. Left to right: an OSM city-map texture as seen from above, checkerboard and concrete textures rendered from the drone camera, and a top-down view of the cubic-spline trajectory for one sequence.

D. Dataset Packaging

The final dataset contains 60 training and 10 test sequences at 100 Hz (1000 camera frames each). For each sequence the generator saves: ground-truth poses and velocities at 1 kHz, noisy IMU readings (gyro + accelerometer) at 1 kHz, camera-frame poses at 100 Hz, pre-computed relative poses (translation and quaternion) between consecutive camera frames, and windowed IMU data between each camera pair (10 samples of 6-D gyro+accel per window). A PyTorch Dataset class loads image pairs, the corresponding IMU window, and the relative-pose label as a single training sample.

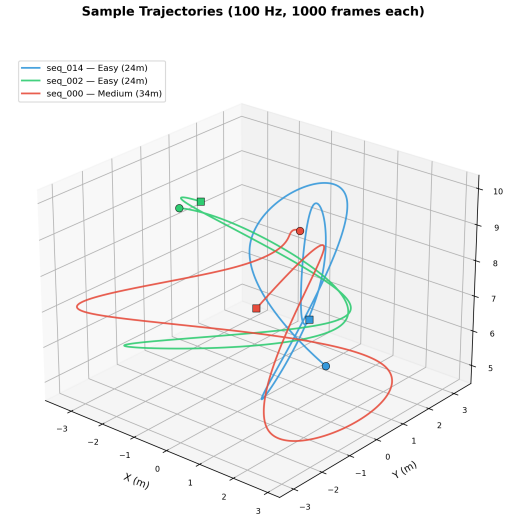


Fig. 2. Three sample trajectories from the training set, color-coded by difficulty. The trajectory generator produces smooth, bounded 3-D paths with varying path lengths.

II. NETWORK ARCHITECTURES

We train three networks that share the same pose-regression interface — each outputs a 7-D vector ($\Delta \mathbf{t}, \Delta \mathbf{q}$) representing the relative translation (3) and unit quaternion (4) between consecutive camera frames — but differ in what sensory input they consume.

A. Vision-Only: *VisionEncoderV2*

The vision-only model stacks two consecutive grayscale frames into a 2-channel input tensor and feeds it through a pretrained ResNet-18 backbone (with the first convolutional layer modified to accept 2 channels). The 512-D feature vector from the global average pool is projected down through a linear layer, passed through dropout, and decoded by a linear pose head into 7 outputs. The quaternion output is ℓ_2 -normalized to enforce unit norm.

Vision-Only: *VisionEncoderV2*

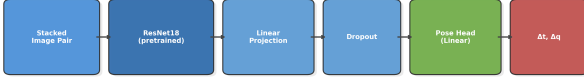


Fig. 3. Vision-only architecture. A pretrained ResNet-18 encodes stacked image pairs; a linear head regresses the relative pose.

B. Inertial-Only: *InertialEncoder*

The inertial-only model takes the 10×6 IMU window (10 samples of 3-axis gyro + 3-axis accel) between consecutive camera frames. The window is treated as a 6-channel 1-D signal and processed by three Conv1d layers with batch normalization and ReLU activations, followed by adaptive average pooling, a linear projection, and the same pose head used by the vision model.

Inertial-Only: *InertialEncoder*



Fig. 4. Inertial-only architecture. Three 1-D convolutional layers process the IMU window; a linear head regresses the relative pose.

C. Visual-Inertial Fusion: *FusionEncoderV2*

The fusion model combines both modalities via late fusion. The vision branch (ResNet-18 + linear projection) and the inertial branch ($3 \times \text{Conv1d}$ + linear projection) each produce a fixed-length embedding. These are concatenated and passed through a two-layer MLP with dropout and ReLU, followed by the pose head.

Visual-Inertial Fusion: *FusionEncoderV2*

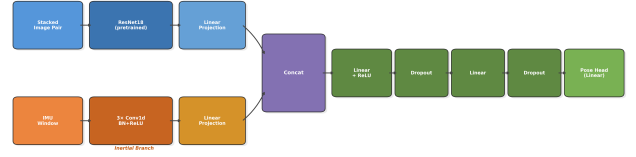


Fig. 5. Visual-inertial fusion architecture. Vision and inertial branches are encoded independently, concatenated, and fused through an MLP before pose regression.

III. TRAINING

All three models are trained to minimize the same loss function, which combines a translation MSE term with a quaternion distance term:

$$\mathcal{L} = \text{MSE}(\Delta \mathbf{t}_{pred}, \Delta \mathbf{t}_{gt}) + \beta \cdot \min(\|\mathbf{q}_{pred} - \mathbf{q}_{gt}\|^2, \|\mathbf{q}_{pred} + \mathbf{q}_{gt}\|^2) \quad (1)$$

The min over $\pm \mathbf{q}$ accounts for the double-cover ambiguity of unit quaternions (i.e., $\mathbf{q}$ and $-\mathbf{q}$ represent the same rotation). We set $\beta = 100$ to heavily weight rotation accuracy, since rotation errors compound quadratically through dead-reckoning while translation errors compound linearly — under-weighting rotation destroys trajectory ATE even when translation loss is low. Training uses Adam with a cosine-annealing learning-rate schedule, gradient clipping at norm 1.0, and batch size 64.

IV. EVALUATION

Each model is evaluated by running inference on every consecutive image pair (and/or IMU window) in a test sequence, collecting the predicted relative poses, and integrating them into a global trajectory via dead-reckoning from the ground-truth initial pose. The resulting trajectory is then aligned to the ground truth using Umeyama’s method (rigid $\text{SE}(3)$ + scale), and the root-mean-square absolute trajectory error (RMSE-ATE) is computed.

V. RESULTS

We trained four versions of the deep VIO pipeline. V1 used a lightweight from-scratch CNN as the vision encoder, but its vision branch suffered mode collapse, predicting nearly constant Δposes regardless of input. In V2 we replaced this with a pretrained ResNet-18 backbone, which leveraged ImageNet priors and fixed the mode-collapse problem. V3 addressed the data side: we generated a new dataset of 60 scenes (up from 30) with much more aggressive camera trajectories, producing roughly $11 \times$ larger per-frame motion (about 3 mm to 34 mm) so the network had more pixel displacement to learn from. V4 kept the V3 setup but added an auxiliary homography prediction head to the vision and fusion networks, taking advantage of the planar-scene geometry of our downward-camera setup. The quantitative results of all four versions are shown in the table below.

The quantitative results of all four versions are shown in the below table.

TABLE I
MEAN RMSE ATE (M) ACROSS PIPELINE VERSIONS. LOWER IS BETTER.
BEST RESULT PER NETWORK SHOWN IN RED; FUSION ROW SHOWN IN BOLD.

Network	V1	V2	V3	V4
Inertial-only	1.89	2.03	1.95	1.95
Vision-only	6.56	4.00	1.82	1.17
Fusion	2.58	2.08	2.45	1.55

VI. DISCUSSION

In V1, the vision-only model diverges badly, it curves away from the ground truth within a few seconds and never recovers, ending up at roughly $3\times$ the ATE of the other two. The inertial model captures the rough shape of the trajectory and stays in the right neighborhood, while fusion sits in between. The pretrained ResNet-18 backbone in V2 improved the vision branch : vision drops from 6.56 m to 1.82 m, actually beating inertial on average. The inertial model is essentially unchanged between versions (1.89 vs 1.95 m), which makes sense since its architecture did not change. Fusion improves slightly (2.58 to 2.45 m).

V3 retrained the V2 architectures on a new dataset with 60 scenes and larger per-frame motion. Vision delivered the biggest single jump of the project, dropping from 4.00 m to 1.82 m (55% improvement); the encoder finally had real pixel displacement to learn from, and the ImageNet-pretrained features carried over from V2 could actually be put to use. Inertial stayed essentially flat at 1.95 m, which is what we expected: the IMU encoder was already extracting whatever signal was in the data, and faster trajectories in image space don't help when the bottleneck is the IMU itself.

The surprise was fusion regressing to 2.45 m, worse than both V2 fusion (2.08 m) and V3 vision-only (1.82 m). Three things were at play. First, fusion was undertrained: we had capped it at 15 epochs based on V2's overfit pattern, but the doubled V3 dataset needed longer training. Second, dropout 0.3 was over-regularizing for the larger sample size. Third, and more interesting, our concat+MLP fusion learns a fixed weighting between modalities at training time. V1 and V2 had unreliable vision, so fusion learned to lean on inertial; when V3 made vision the most reliable modality, fusion could not dynamically reweight at inference, and the inertial branch dragged predictions away from where vision would have placed them.

V4 added an auxiliary homography head to the vision and fusion networks, jointly predicting pose and the 4-corner homography parametrization. The intuition is that our scene — a downward camera looking at a planar floor — is the textbook case where homography is the exact geometric model rather than an approximation, so the network gets a much stronger geometric prior than what ImageNet pretraining alone provides. For fusion we also raised epochs from 15 to 25 and reduced dropout from 0.3 to 0.2, addressing the V3 diagnostics in the same run.

The improvements were substantial across both image-using networks. Vision-only dropped from 1.82 m to 1.17 m (36% improvement), and fusion recovered from V3's regression all the way to 1.55 m (37% improvement over V3, 26% over V2). Notably, vision-only still slightly outperforms fusion, suggesting that even with a homography prior our static-weighting fusion does not fully exploit the now highly-reliable vision modality — which strengthens the case for cross-attention fusion as a follow-up. Inertial-only is unchanged at 1.95 m: V4 inertial reuses the V3 inertial checkpoint without retraining, since IMU has no images and therefore no homography to predict.

VII. CHALLENGES

The vision-only model was by far the hardest to get working. Our reference paper was DeepVIO (Han et al., 2019), which uses a CNN encoder followed by an LSTM for temporal context. We started with a lightweight CNN vision encoder (V1), but it suffered from mode collapse — it would converge to predicting near-zero motion for every frame, which gives a low average loss but useless dead-reckoned trajectories. Switching to a pretrained ResNet-18 backbone (V2) fixed this: the pretrained weights gave the network a strong enough feature extractor that it could actually learn the visual correspondence, bringing vision ATE from 6.56 m down to 1.82 m. However, ResNet-18 has ~ 11 M parameters, which meant we had to double the dataset from 30 to 60 training sequences to avoid overfitting.

We deliberately skipped the LSTM used in DeepVIO. Our task is to predict the relative pose between two consecutive frames given only that pair and the IMU window between them — there is no temporal memory needed. Adding an LSTM would have introduced sequence-level dependencies that complicate training and do not match the frame-pair formulation.

The quaternion loss weighting (β) turned out to matter far more than we expected. Rotation errors compound quadratically through dead-reckoning (a small angular error at frame k rotates every subsequent translation into the wrong direction), while translation errors compound linearly. With $\beta = 1$ the model minimized translation loss just fine but the trajectories diverged within seconds. Only after increasing to $\beta = 100$ did the dead-reckoned trajectories stay coherent. We did not do a formal sweep over β , but the difference between 1 and 100 was night and day.

The fusion model ending up roughly tied with the inertial model (and sometimes worse) was a surprise.

VIII. POSSIBLE RESEARCH FRONTIERS

Future work in the context of our implementation is a better fusion strategy, specifically some form of early fusion.

As research frontiers for this problem, based on our current exposure to deep-learning-based VIO, we propose the following:

- 1) **Replace or embed the dead-reckoning step within the network itself.** If the network predicts with error,

dead-reckoning compounds and cascades that error over time. Bringing dead-reckoning inside the network would let the loss directly penalize trajectory level drift rather than per-frame errors alone.

- 2) **Fuse classical and deep-learning approaches.** Deep learning methods are not yet optimal, even at the SOTA level. A solution that combines classical filtering with learned components a kind of deep declarative network may close the remaining gap.
- 3) **Combine CNN and Transformer architectures.** A hybrid encoder using CNNs for local feature extraction and Transformers for long-range temporal or cross-modal reasoning could give the best of both inductive biases.

IX. CONTRIBUTION

Shibani: Data Generation, Models, Documentation

Ajay: Data Generation, Models, Documentation

X. SAMPLE OUTPUTS PLOTS

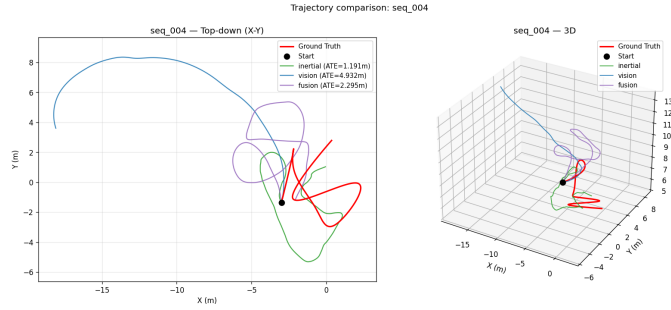


Fig. 6. V1 Sample - 1

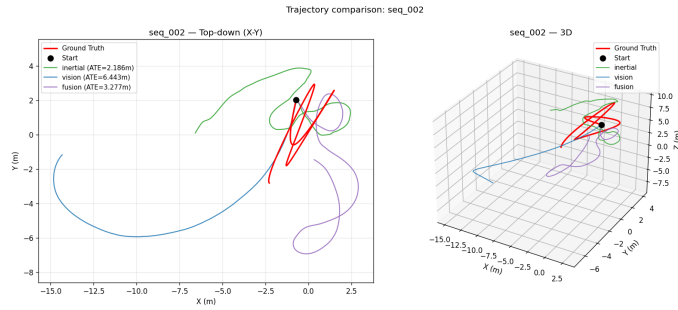


Fig. 7. V1 Sample - 2

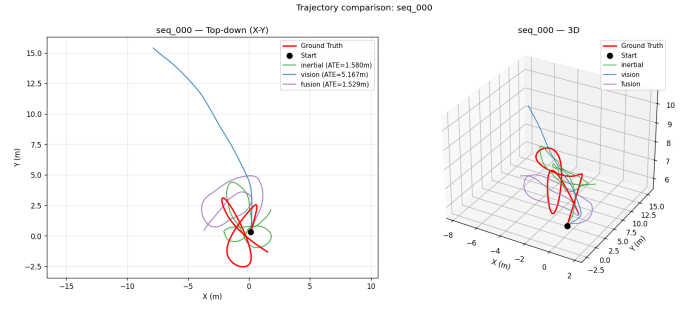


Fig. 8. V2 Sample -1

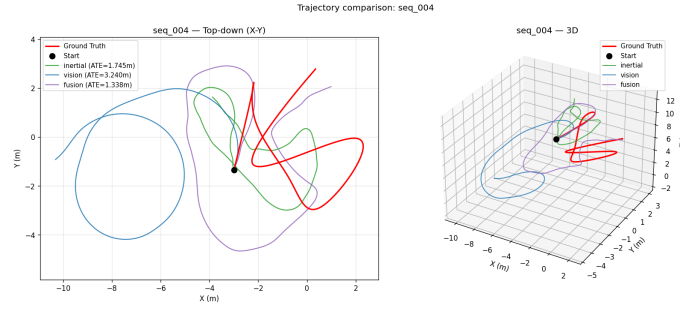


Fig. 9. V2 Sample -2

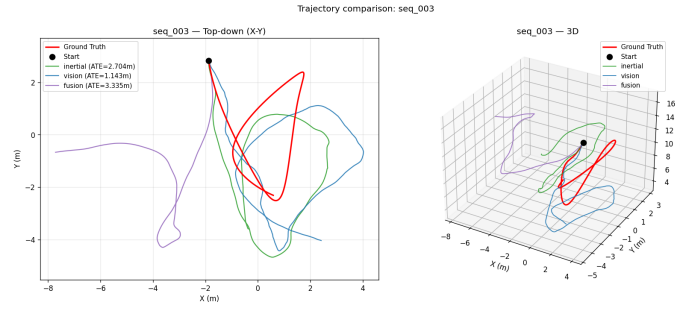


Fig. 10. V3 Sample -1

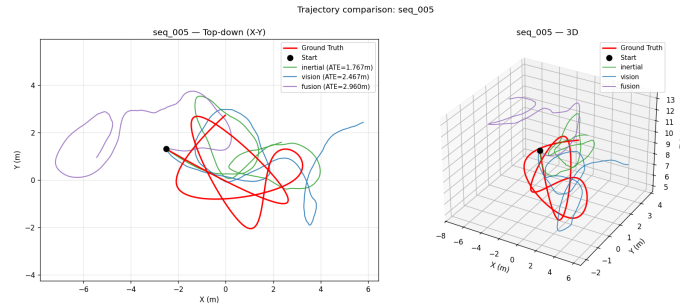


Fig. 11. V3 Sample -2

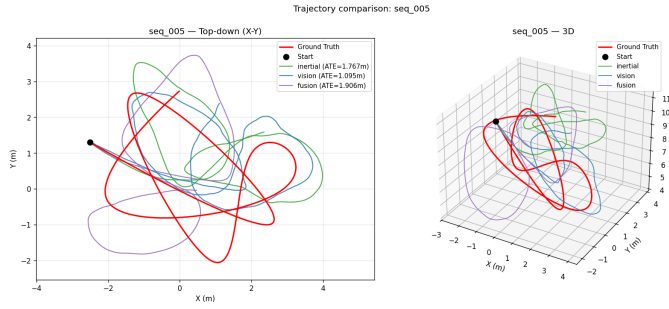


Fig. 12. V3 Sample -1

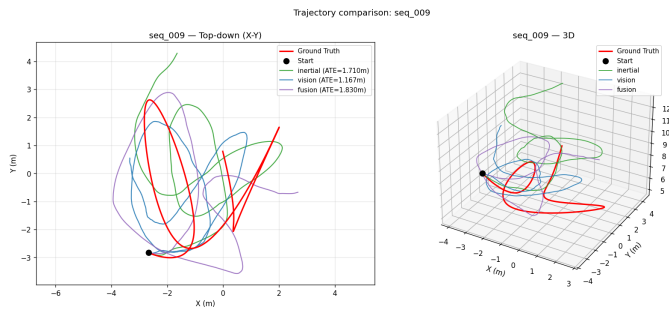


Fig. 13. V3 Sample -2